

Experiment No: 05

Experiment Name: Draw circle using Direct circle drawing algorithm and Mid-point circle drawing Algorithm.

Theory:

The Direct Circle Drawing Algorithm is based on the standard circle equation $(x - x_c)^2 + (y - y_c)^2 = r^2$. By rearranging the equation, the y-coordinate can be found for each x using $y = \sqrt{r^2 - x^2}$. This approach directly uses floating-point arithmetic and trigonometric calculations to determine each pixel point, which makes it accurate but slower for computer graphics.

The Midpoint Circle Drawing Algorithm improves efficiency by using integer arithmetic instead of floating-point calculations. It determines the next pixel position using a decision parameter d that checks whether the midpoint between two possible pixels lies inside or outside the circle boundary. This makes it faster and more suitable for raster displays while maintaining a smooth circular shape.

Code: Direct Circle Drawing Algorithm.

```
1 #include <GL/glut.h>
2 #include <cmath>
3
4 const int WIDTH = 600;
5 const int HEIGHT = 600;
6
7 int xc = 300, yc = 300;
8 int radius = 200;
9
10 void putPixel(int x, int y) {
11     glBegin(GL_POINTS);
12     glVertex2i(x, y);
13     glEnd();
14 }
15
16 void plotCirclePoints(int xc, int yc, int x, int y) {
17     putPixel(xc + x, yc + y);
18     putPixel(xc - x, yc + y);
19     putPixel(xc + x, yc - y);
20     putPixel(xc - x, yc - y);
21     putPixel(xc + y, yc + x);
22     putPixel(xc - y, yc + x);
```

```

23     putPixel(xc + y, yc - x);
24     putPixel(xc - y, yc - x);
25 }
26
27 void drawCircleDirect(int xc, int yc, int r) {
28     int x = 0;
29     int y;
30
31     while (x <= r) {
32         y = (int)round(sqrt(r * r - x * x));
33         plotCirclePoints(xc, yc, x, y);
34         x++;
35     }
36 }
37
38 void drawAxes() {
39     glColor3f(0.0f, 0.0f, 0.0f);
40     glBegin(GL_LINES);
41     glVertex2i(0, yc);
42     glVertex2i(WIDTH, yc);
43     glVertex2i(xc, 0);
44     glVertex2i(xc, HEIGHT);
45     glEnd();
46 }
47
48 void display() {
49     glClear(GL_COLOR_BUFFER_BIT);
50     drawAxes();
51     glColor3f(1.0f, 0.0f, 0.0f);
52     drawCircleDirect(xc, yc, radius);
53     glFlush();
54 }
55
56 void init() {
57     glClearColor(1.0, 1.0, 1.0, 1.0);
58     gluOrtho2D(0, WIDTH, 0, HEIGHT);
59 }
60
61 int main(int argc, char** argv) {
62     glutInit(&argc, argv);
63     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
64     glutInitWindowSize(WIDTH, HEIGHT);
65     glutInitWindowPosition(100, 100);
66     glutCreateWindow("Direct Circle Drawing Algorithm");
67     init();
68     glutDisplayFunc(display);
69     glutMainLoop();
70     return 0;
71 }

```

Output:

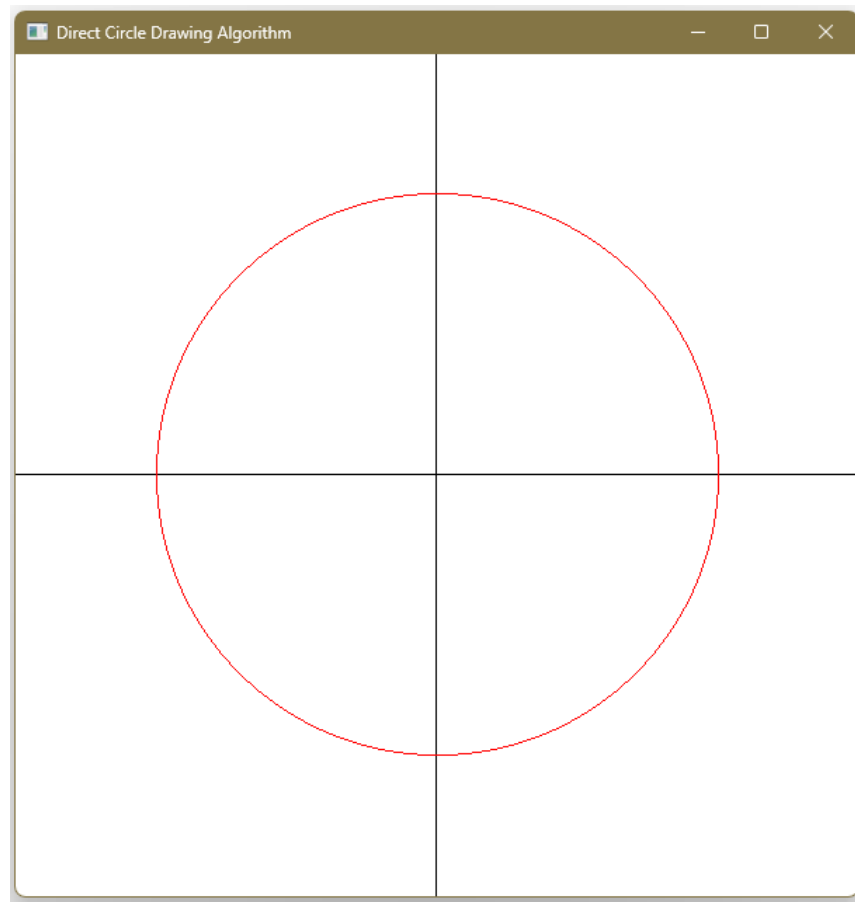


Figure 1: Output of the following code.

Code: Mid-point Circle Drawing Algorithm.

```
1 #include <GL/glut.h>
2
3 const int WIDTH = 600;
4 const int HEIGHT = 600;
5
6 int xc = 300, yc = 300;
7 int radius = 150;
8
9 void putPixel(int x, int y) {
10     glBegin(GL_POINTS);
11     glVertex2i(x, y);
```

```

12     glEnd();
13 }
14
15 void plotCirclePoints(int xc, int yc, int x, int y) {
16     putPixel(xc + x, yc + y);
17     putPixel(xc - x, yc + y);
18     putPixel(xc + x, yc - y);
19     putPixel(xc - x, yc - y);
20     putPixel(xc + y, yc + x);
21     putPixel(xc - y, yc + x);
22     putPixel(xc + y, yc - x);
23     putPixel(xc - y, yc - x);
24 }
25
26 void drawCircleMidpoint(int xc, int yc, int r) {
27     int x = 0;
28     int y = r;
29     int d = 1 - r;
30
31     plotCirclePoints(xc, yc, x, y);
32
33     while (x < y) {
34         if (d < 0) {
35             d = d + 2 * x + 3;
36         } else {
37             d = d + 2 * (x - y) + 5;
38             y--;
39         }
40         x++;
41         plotCirclePoints(xc, yc, x, y);
42     }
43 }
44
45 void drawAxes() {
46     glColor3f(0.0f, 0.0f, 0.0f);
47     glBegin(GL_LINES);
48     glVertex2i(0, yc);
49     glVertex2i(WIDTH, yc);
50     glVertex2i(xc, 0);
51     glVertex2i(xc, HEIGHT);
52     glEnd();
53 }
54
55 void display() {
56     glClear(GL_COLOR_BUFFER_BIT);
57     drawAxes();
58     glColor3f(1.0f, 0.0f, 0.0f);
59     drawCircleMidpoint(xc, yc, radius);
60     glFlush();
61 }
62
63 void init() {

```

```

64     glClearColor(1.0, 1.0, 1.0, 1.0);
65     gluOrtho2D(0, WIDTH, 0, HEIGHT);
66 }
67
68 int main(int argc, char** argv) {
69     glutInit(&argc, argv);
70     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
71     glutInitWindowSize(WIDTH, HEIGHT);
72     glutInitWindowPosition(100, 100);
73     glutCreateWindow("Midpoint Circle Drawing Algorithm");
74     init();
75     glutDisplayFunc(display);
76     glutMainLoop();
77     return 0;
78 }

```

Output:

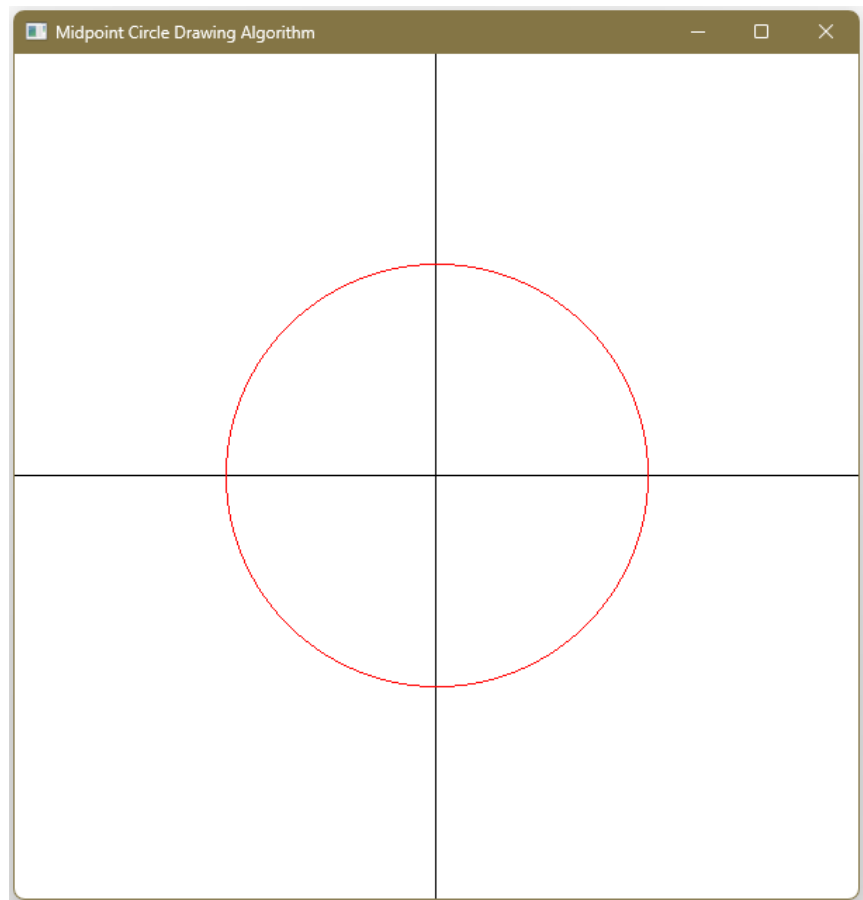


Figure 2: Output of the following code.

Conclusion:

Both algorithms were successfully implemented to draw circles. The direct method provided a clear circular shape but involved more computation due to square roots and floating-point operations. The midpoint algorithm produced the same result with less computation time and only integer operations, proving it to be more efficient and practical for real-time graphics applications.